

- Lab Draken - Proxmox, OPNsense et Infrastructure as Code
 - Introduction
 - Contexte professionnel
 - Technologies Utilisées
 - Architecture du Réseau
 - Tableau du plan d'adressage IP
 - Schéma ASCII de la topologie
 - Configuration des Équipements
 - 1. Configuration d'OPNsense comme routeur du lab
 - 2. Création du bastion IaC (draken-admin)
 - 3. Création de modules Terraform réutilisables
 - 4. Création des segments AD et DMZ
 - 5. Rattachement des segments à OPNsense
 - 6. Validation finale
 - Tests et Résultats
 - Difficultés Rencontrées et Solutions
 - Améliorations Possibles
 - Ce que ce Projet Démonstre
 - Références et Ressources

Lab Draken - Proxmox, OPNsense et Infrastructure as Code

Mise en place d'un lab self-hébergé avec Proxmox VE, OPNsense comme pare-feu et routeur, et Infrastructure as Code via Terraform et Ansible.

Auteur : Garlens Charles

Formation : BTS SIO – Option SISR

Année : 2025-2026

Introduction

Dans le cadre de mon BTS SIO option SISR, j'ai décidé de migrer mon ancien serveur physique vers un vrai environnement virtualisé, avec Proxmox VE comme hyperviseur et OPNsense comme pare-feu/routeur. Je me suis fixé un objectif volontairement ambitieux : ne rien configurer à la main quand c'était évitable, et piloter toute l'infrastructure (jusqu'au réseau lui-même) avec Terraform et Ansible. Ce document reprend ce que j'ai mis en place, comment, et ce que j'ai appris en le faisant.

Contexte professionnel

En travaillant sur ce lab, j'ai compris pourquoi ce type d'architecture est autant utilisé en entreprise : - **Sécurité du SI** : un pare-feu qui filtre entre plusieurs zones réseau limite ce qu'un attaquant peut atteindre s'il compromet une seule machine. - **Gestion des identités** : séparer le réseau des services (LAN) de celui des futurs comptes/annuaire (AD) évite qu'un problème sur l'un affecte l'autre. - **Conformité réglementaire** : documenter précisément qui a accès à quoi, et pourquoi, c'est ce qu'on attend en audit. - **Réduction de la surface d'attaque** : une zone DMZ permet d'isoler ce qui doit être exposé, sans donner un accès direct au reste du réseau. - **Traçabilité** : avec l'Infrastructure as Code, chaque changement d'infra passe par un fichier versionné — je peux savoir qui a changé quoi et quand. - **Continuité de service** : automatiser le déploiement (modules Terraform) permet de recréer une machine cassée en quelques minutes plutôt qu'en repartant de zéro.

Technologies Utilisées

- Proxmox VE 9.2.2
— hyperviseur pour la virtualisation des serveurs.
- OPNsense

- pare-feu et routeur pour la gestion du trafic réseau.
- Terraform
 - outil d'Infrastructure as Code pour la gestion des ressources virtuelles.
- Ansible
 - outil de configuration et de déploiement pour les serveurs et applications.
- Debian 13
 - système d'exploitation pour les machines virtuelles et conteneurs.
- Docker
 - pour l'exécution des applications dans des conteneurs.

Architecture du Réseau

La topologie du réseau est composée de quatre segments : -

WAN (vmbr0)

: connecté à Internet via la box Internet. -

LAN (vmbr1)

: réseau local pour les machines virtuelles et les conteneurs. -

AD (vmbr2)

: réseau pour les services d'annuaire et d'authentification. -

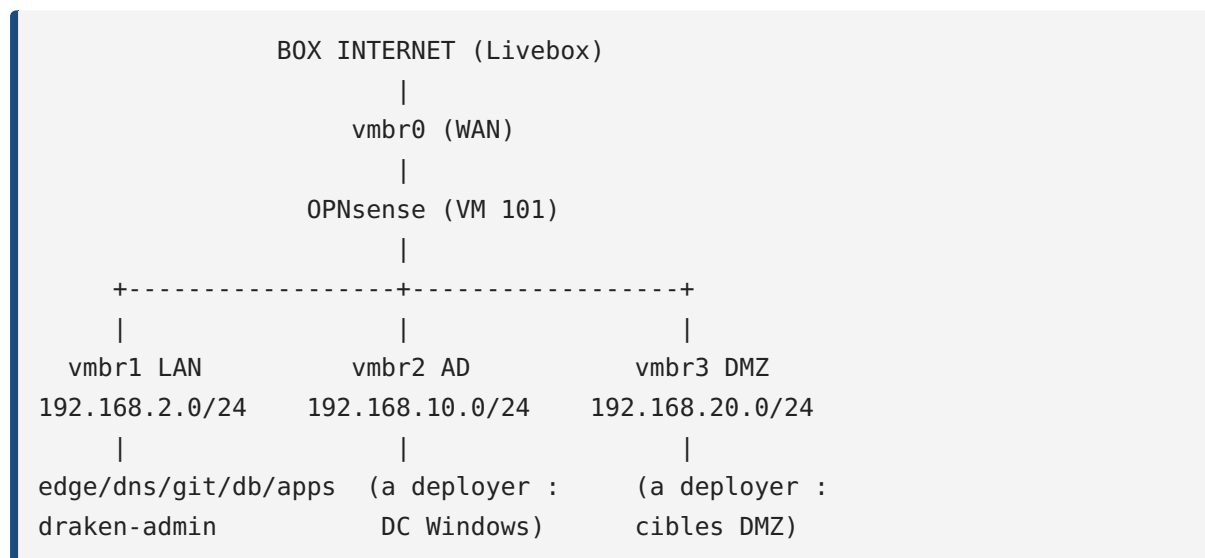
DMZ (vmbr3)

: zone démilitarisée pour les services exposés à Internet.

Tableau du plan d'adressage IP

Équipement	Adresse IP	Rôle
OPNsense (LAN)	192.168.2.1	Gateway pare-feu et routeur du segment LAN
OPNsense (AD)	192.168.10.1	Gateway pare-feu et routeur du segment AD
OPNsense (DMZ)	192.168.20.1	Gateway pare-feu et routeur du segment DMZ
draken-admin (CT 102)	192.168.2.101	Bastion pour Terraform et Ansible
edge (CT 110)	192.168.2.10	Reverse proxy Traefik
dns (CT 111)	192.168.2.11	Pi-hole (arrêté volontairement)
git (CT 112)	192.168.2.12	Gitea
db (CT 113)	192.168.2.13	PostgreSQL
apps (VM 120)	192.168.2.20	Docker, services applicatifs

Schéma ASCII de la topologie



Configuration des Équipements

1. Configuration d'OPNsense comme routeur du lab

J'ai commencé par installer OPNsense en VM (101) avec deux interfaces :

WAN (vmbr0)

, branchée sur la carte physique du serveur, et

LAN (vmbr1)

, mon réseau interne.

```
qm set 101 -net0 virtio,bridge=vmbr0
qm set 101 -net1 virtio,bridge=vmbr1
```

Résultat : OPNsense est joignable sur

192.168.2.1

Pourquoi cette étape est importante ? C'est la base de tout le reste : si le routeur/pare-feu n'est pas en place en premier, je ne peux ni segmenter le réseau ni contrôler ce qui circule entre mes futures zones. C'est aussi la première fois que je

faisais tourner un pare-feu dédié en frontal, plutôt que de laisser la box internet tout gérer.

2. Création du bastion IaC (draken-admin)

Pour ne pas piloter mon infra depuis mon PC perso (clé API qui traîne, environnement pas reproductible), j'ai créé un LXC dédié,

```
draken-admin
```

, qui devient mon unique point d'entrée pour Terraform et Ansible.

```
# providers.tf – déclare le provider bpg/proxmox
provider "proxmox" {
  endpoint = var.proxmox_endpoint
  api_token = var.proxmox_api_token
}

# variables.tf – tout ce qui est partagé entre mes ressources
variable "node_name" {}

# outputs.tf – ce que j'expose pour le réutiliser ailleurs
output "vm_id" { value = proxmox_virtual_environment_vm.this.vm_id }
```

Résultat : le bastion est prêt, et surtout mon token API n'existe que dans une variable d'environnement sur ce LXC — jamais dans un fichier que je pousse sur Gitea.

Pourquoi cette étape est importante ? J'aurais pu lancer Terraform depuis n'importe quelle machine, mais avoir un bastion dédié m'oblige à garder mes secrets au même endroit et évite qu'ils traînent sur mon poste de travail.

3. Création de modules Terraform réutilisables

Au début je copiais-collais un fichier Terraform par service (edge.tf, dns.tf, git.tf...), ce qui devenait vite ingérable dès que je voulais changer un paramètre commun. J'ai donc écrit un module

```
lxc
```

générique et un module

```
vm
```

(cloud-init Debian 13), que j'appelle ensuite pour chaque service.

```
module "edge" {
  source      = "../modules/lxc"
  vm_id       = 110
  bridge      = "vmbr1"
  ip_address  = "192.168.2.10/24"
}
```

Résultat : un nouveau service se déploie en quelques lignes de configuration, sans dupliquer de code.

Pourquoi cette étape est importante ? C'est le moment où j'ai compris l'intérêt réel des modules Terraform : au-delà du confort, ça évite qu'un oubli de copier-coller crée une incohérence entre deux services qui devraient être configurés pareil.

4. Création des segments AD et DMZ

Je pensais au départ que Terraform ne pouvait gérer que des VM/LXC, pas le réseau lui-même. En creusant la documentation officielle du provider `bpg`, j'ai trouvé la ressource qu'il me fallait : un module

```
network
```

dédié qui crée les bridges

```
vmbr2
```

(AD) et

```
vmbr3
```

(DMZ), avec

```
ports = []
```

pour un bridge isolé sans carte physique.

```
resource "proxmox_virtual_environment_network_linux_bridge" "ad" {
  node_name = var.node_name
  name      = "vmbr2"
  ports     = []
  autostart = true
}
```

J'ai volontairement laissé

```
vmbr0
```

```
/
```

```
vmbr1
```

hors de ce module : ils existaient déjà avant que je commence à utiliser Terraform sur ce projet, et je ne voulais pas risquer de les casser avec un drift de state sur un lab qui doit rester joignable. Résultat vérifié : après

```
terraform apply
```

, les bridges

```
vmbr2
```

et

```
vmbr3
```

sont actifs sur le nœud (confirmé par

```
ip a
```

).

Pourquoi cette étape est importante ? Cette étape m'a fait réaliser que l'Infrastructure as Code, ce n'est pas juste des VM : le réseau qui les relie peut, et à mon avis doit, être versionné aussi — sinon la moitié de l'infra reste non documentée.

5. Rattachement des segments à OPNsense

Il fallait ensuite donner à OPNsense deux nouvelles cartes réseau virtuelles pointant vers

```
vmbr2
```

```
/
```

```
vmbr3
```

, puis les assigner et leur donner une IP statique (

```
.1
```

) depuis l'interface OPNsense.

```
qm set 101 -net2 virtio,bridge=vmbr2
qm set 101 -net3 virtio,bridge=vmbr3
qm stop 101
qm start 101
```

Résultat : les segments sont rattachés à OPNsense — mais pas du premier coup (voir la difficulté rencontrée ci-dessous).

Pourquoi cette étape est importante ? Ça a été mon premier vrai blocage du projet : un simple

```
qm reboot
```

ne faisait pas apparaître les nouvelles cartes réseau. J'ai dû comprendre qu'un redémarrage logiciel ne suffit pas toujours, et qu'un arrêt complet force une réénumération du bus PCI virtuel.

6. Validation finale

Pour finir, j'ai voulu vérifier moi-même que tout tournait vraiment, plutôt que de me fier au fait que

```
terraform apply
```

n'avait renvoyé aucune erreur.

```
qm list
pct list
ip a | grep vmbr
```

Résultat : toutes les VM/LXC attendus sont bien

```
running
```

(sauf

```
dns
```

, que j'ai arrêté volontairement) et les 4 bridges sont présents.

Pourquoi cette étape est importante ? Un

```
terraform apply
```

qui réussit ne veut pas dire que le service tourne réellement derrière — je l'ai vérifié pour de vrai en allant lire l'état sur le nœud, pas seulement la sortie de Terraform.

Tests et Résultats

Test	Commande	Résultat
Vérification de l'état des machines virtuelles	<pre>qm list</pre>	 OPNsense et apps affichés running
Vérification de l'état des conteneurs	<pre>pct list</pre>	 draken-admin/edge/git/db running, dns stopped (volontaire)
Vérification de la présence des bridges	<pre>ip a \ grep vmbr</pre>	 vmbr0-3 tous présents, y compris les 2 créés par Terraform
Vérification du reverse proxy	<pre>ps aux \ grep traefik</pre> (LXC edge)	 processus Traefik actif sur les entrypoints 80/443
Vérification post-terraform apply du module réseau	<pre>ip a</pre> après apply	 puis après <pre>qm stop</pre> / <pre>qm start</pre> (réénumération PCI)

Difficultés Rencontrées et Solutions

#	Problème	Cause	Solution
1	<code>qm reboot 101</code> insuffisant pour faire apparaître <code>net2/net3</code>	Redémarrage logiciel n'force pas la rénumération PCI	<code>qm stop 101</code> puis <code>qm start 101</code>
2	Hypothèse erronée : Terraform ne gèrerait que VM/LXC, pas le réseau	Méconnaissance du provider bpg	Documentation officielle bpg confirme la ressource <code>network_linux_bridge</code>
3	Risque si <code>vmbr0/vmbr1</code> importés dans le state Terraform	Drift de state → recréation destructrice possible	Terraform gère uniquement le nouveau (<code>vmbr2/vmbr3</code>) , le reste en manuel

Améliorations Possibles

Sécurité renforcée - Implémenter des règles de pare-feu

`OPNsense`

d'isolation stricte

`AD→LAN`

et

`DMZ→LAN`

- Authentification devant le dashboard

`Traefik`

(actuellement insecure, interne)

Infrastructure étendue - Déployer les VM du lab

AD

(DC Windows + client) par clonage

Terraform

d'un template sysprep - Cibles

DMZ

(service vulnérable, honeypot) pour exercices d'intrusion isolés -

DHCP

par segment sur

OPNsense

(actuellement IP statiques)

Supervision - LXC

Ollama

pour dashboards perf/pare-feu/intrusion

Ce que ce Projet Démontre

Compétence	Description
Virtualisation	Administration Proxmox VE en prod (VM + LXC)
Sécurité réseau	Pare-feu OPNsense, segmentation 4 zones, blocage par défaut
Infrastructure as Code	Terraform (réseau inclus) + Ansible, secrets hors code
Architecture modulaire	Modules Terraform réutilisables (lxc, vm)
Diagnostic	Identification qu'un reboot logiciel ne suffit pas à une rénumération PCI
Reverse proxy	Traefik comme point d'entrée du lab

Références et Ressources

- Documentation officielle Proxmox VE
- Documentation officielle OPNsense
- Documentation officielle Terraform
- Documentation officielle Ansible
- Provider bpg pour Proxmox VE sur Terraform